

Algoritmos - Actividad Guiada 1

Nombre: Guido Quispe Quispe

http: https://github.com/Guidouib/Algoritmos_de_Optimizacion/tree/main/AG1-%20Actividad%20Guiada%201

Torres de Hanoi con Divide y vencerás

```
In [1]: def Torres_Hanoi(N, desde, hasta):
    if N ==1 :
        print("Lleva la ficha " ,desde , " hasta " , hasta )

    else:
        #Torres_Hanoi(N-1, desde, 6-desde-hasta )
        Torres_Hanoi(N-1, desde, 6-desde-hasta )
        print("Lleva la ficha " ,desde , " hasta " , hasta )
        #Torres_Hanoi(N-1,6-desde-hasta, hasta )
        Torres_Hanoi(N-1, 6-desde-hasta , hasta )

Torres_Hanoi(3, 1 , 3)
```

```
Lleva la ficha 1 hasta 3
Lleva la ficha 1 hasta 2
Lleva la ficha 3 hasta 2
Lleva la ficha 1 hasta 3
Lleva la ficha 2 hasta 1
Lleva la ficha 2 hasta 3
Lleva la ficha 1 hasta 3
```

```
In [2]: #Sucesión de Fibonacci
#https://es.wikipedia.org/wiki/Sucesi%C3%B3n_de_Fibonacci
#Calculo del termino n-simo de la sucesión de Fibonacci
def Fibonacci(N:int):
    if N < 2:
        return 1
    else:
        return Fibonacci(N-1)+Fibonacci(N-2)

Fibonacci(5)
```

Out[2]: 8

Devolución de cambio por técnica voraz

```
In [3]: def cambio_monedas(N, SM):
    SOLUCION = [0]*len(SM)    #SOLUCION = [0,0,0,0,..]
    ValorAcumulado = 0

    for i,valor in enumerate(SM):
        monedas = (N-ValorAcumulado)//valor
        SOLUCION[i] = monedas
        ValorAcumulado = ValorAcumulado + monedas*valor
```

```

    if ValorAcumulado == N:
        return SOLUCION

cambio_monedas(15,[25,10,5,1])

```

Out[3]: [0, 1, 1, 0]

N-Reinas por técnica de vueta atrás

```

In [4]: def describe(S):
    n = len(S)
    for x in range(n):
        print("")
        for i in range(n):
            if S[i] == x+1:
                print(" X ", end="")
            else:
                print(" - ", end="")

def es_prometedora(SOLUCION,etapa):
    #print(SOLUCION)
    #Si la solución tiene dos valores iguales no es valida => Dos reinas en la mis
    for i in range(etapa+1):
        #print("El valor " + str(SOLUCION[i]) + " está " + str(SOLUCION.count(SOLUCION[i]))
        if SOLUCION.count(SOLUCION[i]) > 1:
            return False

        #Verifica las diagonales
        for j in range(i+1, etapa + 1 ):
            #print("Comprobando diagonal de " + str(i) + " y " + str(j))
            if abs(i-j) == abs(SOLUCION[i]-SOLUCION[j]) : return False
    return True

def reinas(N, solucion=[], etapa=0):
    if len(solucion) == 0:
        solucion=[0 for i in range(N)]

    for i in range(1, N+1):
        solucion[etapa] = i

        if es_prometedora(solucion, etapa):
            if etapa == N-1:
                print(solucion)
                #describe(solucion)
                print()
            else:
                reinas(N, solucion, etapa+1)
        else:
            None

        solucion[etapa] = 0

reinas(8)

```

[1, 5, 8, 6, 3, 7, 2, 4]

[1, 6, 8, 3, 7, 4, 2, 5]

[1, 7, 4, 6, 8, 2, 5, 3]

[1, 7, 5, 8, 2, 4, 6, 3]

[2, 4, 6, 8, 3, 1, 7, 5]

[2, 5, 7, 1, 3, 8, 6, 4]

[2, 5, 7, 4, 1, 8, 6, 3]

[2, 6, 1, 7, 4, 8, 3, 5]

[2, 6, 8, 3, 1, 4, 7, 5]

[2, 7, 3, 6, 8, 5, 1, 4]

[2, 7, 5, 8, 1, 4, 6, 3]

[2, 8, 6, 1, 3, 5, 7, 4]

[3, 1, 7, 5, 8, 2, 4, 6]

[3, 5, 2, 8, 1, 7, 4, 6]

[3, 5, 2, 8, 6, 4, 7, 1]

[3, 5, 7, 1, 4, 2, 8, 6]

[3, 5, 8, 4, 1, 7, 2, 6]

[3, 6, 2, 5, 8, 1, 7, 4]

[3, 6, 2, 7, 1, 4, 8, 5]

[3, 6, 2, 7, 5, 1, 8, 4]

[3, 6, 4, 1, 8, 5, 7, 2]

[3, 6, 4, 2, 8, 5, 7, 1]

[3, 6, 8, 1, 4, 7, 5, 2]

[3, 6, 8, 1, 5, 7, 2, 4]

[3, 6, 8, 2, 4, 1, 7, 5]

[3, 7, 2, 8, 5, 1, 4, 6]

[3, 7, 2, 8, 6, 4, 1, 5]

[3, 8, 4, 7, 1, 6, 2, 5]

[4, 1, 5, 8, 2, 7, 3, 6]

[4, 1, 5, 8, 6, 3, 7, 2]

[4, 2, 5, 8, 6, 1, 3, 7]

[4, 2, 7, 3, 6, 8, 1, 5]

[4, 2, 7, 3, 6, 8, 5, 1]

[4, 2, 7, 5, 1, 8, 6, 3]

[4, 2, 8, 5, 7, 1, 3, 6]

[4, 2, 8, 6, 1, 3, 5, 7]

[4, 6, 1, 5, 2, 8, 3, 7]

[4, 6, 8, 2, 7, 1, 3, 5]

[4, 6, 8, 3, 1, 7, 5, 2]

[4, 7, 1, 8, 5, 2, 6, 3]

[4, 7, 3, 8, 2, 5, 1, 6]

[4, 7, 5, 2, 6, 1, 3, 8]

[4, 7, 5, 3, 1, 6, 8, 2]

[4, 8, 1, 3, 6, 2, 7, 5]

[4, 8, 1, 5, 7, 2, 6, 3]

[4, 8, 5, 3, 1, 7, 2, 6]

[5, 1, 4, 6, 8, 2, 7, 3]

[5, 1, 8, 4, 2, 7, 3, 6]

[5, 1, 8, 6, 3, 7, 2, 4]

[5, 2, 4, 6, 8, 3, 1, 7]

[5, 2, 4, 7, 3, 8, 6, 1]

[5, 2, 6, 1, 7, 4, 8, 3]

[5, 2, 8, 1, 4, 7, 3, 6]

[5, 3, 1, 6, 8, 2, 4, 7]

[5, 3, 1, 7, 2, 8, 6, 4]

[5, 3, 8, 4, 7, 1, 6, 2]

[5, 7, 1, 3, 8, 6, 4, 2]

[5, 7, 1, 4, 2, 8, 6, 3]

[5, 7, 2, 4, 8, 1, 3, 6]

[5, 7, 2, 6, 3, 1, 4, 8]

[5, 7, 2, 6, 3, 1, 8, 4]

[5, 7, 4, 1, 3, 8, 6, 2]

[5, 8, 4, 1, 3, 6, 2, 7]

[5, 8, 4, 1, 7, 2, 6, 3]

[6, 1, 5, 2, 8, 3, 7, 4]

[6, 2, 7, 1, 3, 5, 8, 4]

[6, 2, 7, 1, 4, 8, 5, 3]

[6, 3, 1, 7, 5, 8, 2, 4]

[6, 3, 1, 8, 4, 2, 7, 5]

[6, 3, 1, 8, 5, 2, 4, 7]

[6, 3, 5, 7, 1, 4, 2, 8]

[6, 3, 5, 8, 1, 4, 2, 7]

[6, 3, 7, 2, 4, 8, 1, 5]

[6, 3, 7, 2, 8, 5, 1, 4]

[6, 3, 7, 4, 1, 8, 2, 5]

[6, 4, 1, 5, 8, 2, 7, 3]

[6, 4, 2, 8, 5, 7, 1, 3]

[6, 4, 7, 1, 3, 5, 2, 8]

[6, 4, 7, 1, 8, 2, 5, 3]

[6, 8, 2, 4, 1, 7, 5, 3]

[7, 1, 3, 8, 6, 4, 2, 5]

[7, 2, 4, 1, 8, 5, 3, 6]

[7, 2, 6, 3, 1, 4, 8, 5]

[7, 3, 1, 6, 8, 5, 2, 4]

[7, 3, 8, 2, 5, 1, 6, 4]

[7, 4, 2, 5, 8, 1, 3, 6]

[7, 4, 2, 8, 6, 1, 3, 5]

[7, 5, 3, 1, 6, 8, 2, 4]

[8, 2, 4, 1, 7, 5, 3, 6]

[8, 2, 5, 3, 1, 7, 4, 6]

[8, 3, 1, 6, 2, 5, 7, 4]

[8, 4, 1, 3, 6, 2, 7, 5]

Viaje por el rio. Programación dinámica

```
In [5]: TARIFAS = [
    [0,5,4,3,999,999,999],
    [999,0,999,2,3,999,11],
    [999,999, 0,1,999,4,10],
    [999,999,999, 0,5,6,9],
    [999,999, 999,999,0,999,4],
    [999,999, 999,999,999,0,3],
    [999,999,999,999,999,999,0]
    ]

#####
def Precios(TARIFAS):
#####
    #Total de Nodos
    N = len(TARIFAS[0])

    #Inicialización de la tabla de precios
    PRECIOS = [ [9999]*N for i in [9999]*N]
    RUTA = [ [""]*N for i in [""]*N]

    for i in range(0,N-1):
        RUTA[i][i] = i                #Para ir de i a i se "pasa por i"
        PRECIOS[i][i] = 0             #Para ir de i a i se se paga 0
        for j in range(i+1, N):
            MIN = TARIFAS[i][j]
            RUTA[i][j] = i

            for k in range(i, j):
                if PRECIOS[i][k] + TARIFAS[k][j] < MIN:
                    MIN = min(MIN, PRECIOS[i][k] + TARIFAS[k][j] )
                    RUTA[i][j] = k                #Anota que para ir de i a j hay que pasar po
                PRECIOS[i][j] = MIN

    return PRECIOS,RUTA
#####

PRECIOS,RUTA = Precios(TARIFAS)
#print(PRECIOS[0][6])

print("PRECIOS")
for i in range(len(TARIFAS)):
    print(PRECIOS[i])

print("\nRUTA")
for i in range(len(TARIFAS)):
    print(RUTA[i])

#Determinar la ruta con Recursividad
def calcular_ruta(RUTA, desde, hasta):
```

```

if desde == hasta:
    #print("Ir a :" + str(desde))
    return ""
else:
    return str(calcular_ruta( RUTA, desde, RUTA[desde][hasta])) + \
           ',' + \
           str(RUTA[desde][hasta] \
              )

print("\nLa ruta es:")
calcular_ruta(RUTA, 0,6)

```

PRECIOS

```

[0, 5, 4, 3, 8, 8, 11]
[9999, 0, 999, 2, 3, 8, 7]
[9999, 9999, 0, 1, 6, 4, 7]
[9999, 9999, 9999, 0, 5, 6, 9]
[9999, 9999, 9999, 9999, 0, 999, 4]
[9999, 9999, 9999, 9999, 9999, 0, 3]
[9999, 9999, 9999, 9999, 9999, 9999, 9999]

```

RUTA

```

[0, 0, 0, 0, 1, 2, 5]
['', 1, 1, 1, 1, 3, 4]
['', '', 2, 2, 3, 2, 5]
['', '', '', 3, 3, 3, 3]
['', '', '', '', 4, 4, 4]
['', '', '', '', '', 5, 5]
['', '', '', '', '', '', '']

```

La ruta es:

Out[5]: ',0,2,5'

Actividades extra

Problema: Encontrar los dos puntos más cercanos

- Dado un conjunto de puntos se trata de encontrar los dos puntos más cercanos

Guía para aprendizaje:

- Suponer en 1D, o sea, una lista de números: [3403, 4537, 9089, 9746, 7259,]
- Primer intento: Fuerza bruta
- Calcular la complejidad. ¿Se puede mejorar?
- Segundo intento. Aplicar Divide y Vencerás
- Calcular la complejidad. ¿Se puede mejorar?
- Extender el algoritmo a 2D: [(1122, 6175), (135, 4076), (7296, 2741)...]
- Extender el algoritmo a 3D.

```

In [6]: def Fuerza_bruta(pts):
        best = float('inf')
        pair = None
        for i in range(len(pts)):
            for j in range(i+1, len(pts)):
                d = abs(pts[i] - pts[j])
                if d < best:

```

```

        best, pair = d, (pts[i], pts[j])
    return pair, best
pair, dist = Fuerza_bruta([1, 4, 9, 10, 20])
assert dist == 1
assert set(pair) == {9, 10}

```

Calcular la complejidad. ¿Se puede mejorar? si se puede mejorar, ordenando y revisar los pares consecutivos.

```

In [7]: def closest_sorted(pts):
        pts = sorted(pts)           # O(n Log n)
        best = float('inf')
        pair = None
        for i in range(len(pts)-1): # O(n)
            d = pts[i+1] - pts[i]
            if d < best:
                best, pair = d, (pts[i], pts[i+1])
        return pair, best

```

- Segundo intento. Aplicar Divide y Vencerás

```

In [8]: def closest_dc(pts):
        pts = sorted(pts)           # O(n Log n)
        return _dc(pts)

def _dc(pts):
    n = len(pts)

    # Caso base: 1 o 2 elementos
    if n == 2:
        return (pts[0], pts[1]), pts[1] - pts[0]
    if n == 1:
        return None, float('inf')

    # Dividir
    mid = n // 2
    left_pair, left_dist = _dc(pts[:mid])
    right_pair, right_dist = _dc(pts[mid:])

    # Mejor de las dos mitades
    if left_dist <= right_dist:
        best_pair, best_dist = left_pair, left_dist
    else:
        best_pair, best_dist = right_pair, right_dist

    # Cruce: comparar el elemento del borde izquierdo con el derecho
    cross_dist = pts[mid] - pts[mid - 1]
    if cross_dist < best_dist:
        best_pair, best_dist = (pts[mid - 1], pts[mid]), cross_dist

    return best_pair, best_dist

```

```

In [9]: import random

def test_vs_brute(n=100, runs=500):
    for _ in range(runs):
        pts = random.sample(range(10000), n)

```

```

_, dist_brute = Fuerza_bruta(pts)
_, dist_dc = closest_dc(pts)
assert dist_brute == dist_dc, f"Diferencia: {dist_brute} vs {dist_dc} en
print(f"{runs} pruebas aleatorias pasaron")

test_vs_brute()

```

500 pruebas aleatorias pasaron

- Extender el algoritmo a 2D: [(1122, 6175), (135, 4076), (7296, 2741)...]

Divide y Vencerás closest_dc_2d Tres pasos:

1. Dividir — ordenar por x y partir por la mediana
2. Resolver — llamada recursiva en mitad izquierda y derecha
3. Combinar — revisar solo los puntos en una franja de ancho 2δ alrededor de la división, ordenados por y, con a lo sumo 7 vecinos cada uno

Complejidad: $O(n \log^2 n)$.

```

In [10]: import math
import random

def distancia(p, q):
    return math.sqrt((p[0]-q[0])**2 + (p[1]-q[1])**2)

# Fuerza bruta 2D

def closest_brute_2d(pts):
    best = float('inf')
    pair = None
    for i in range(len(pts)):
        for j in range(i+1, len(pts)):
            d = distancia(pts[i], pts[j])
            if d < best:
                best, pair = d, (pts[i], pts[j])
    return pair, best

# Divide y Vencerás 2D

def closest_dc_2d(pts):
    pts_x = sorted(pts, key=lambda p: p[0])
    return _dc2d(pts_x)

def _dc2d(pts_x):
    n = len(pts_x)

    # Caso base: fuerza bruta para n pequeño
    if n <= 3:
        return closest_brute_2d(pts_x)

    # Dividir por la mediana en x
    mid = n // 2
    mid_x = pts_x[mid][0]

    left_pair, left_dist = _dc2d(pts_x[:mid])
    right_pair, right_dist = _dc2d(pts_x[mid:])

```

```

# Mejor de Las dos mitades
if left_dist <= right_dist:
    best_pair, delta = left_pair, left_dist
else:
    best_pair, delta = right_pair, right_dist

franja = [p for p in pts_x if abs(p[0] - mid_x) < delta]

# Ordenar La franja por y
franja.sort(key=lambda p: p[1])

# Comparar cada punto con Los siguientes (a lo sumo 7 vecinos por geometría)
for i in range(len(franja)):
    j = i + 1
    while j < len(franja) and (franja[j][1] - franja[i][1]) < delta:
        d = distancia(franja[i], franja[j])
        if d < delta:
            delta = d
            best_pair = (franja[i], franja[j])
        j += 1

return best_pair, delta

# EJEMPLO
pts = [(1122, 6175), (135, 4076), (7296, 2741)]
pair, dist = closest_dc_2d(pts)
print(f"Par más cercano: {pair}")
print(f"Distancia: {dist:.2f}")

```

Par más cercano: ((135, 4076), (1122, 6175))

Distancia: 2319.48

- Extender el algoritmo a 3D.
- solo cambia la distancia y el número máximo de vecinos en la franja.

```

In [11]: import math
import random

# Utilidad

def distancia_3d(p, q):
    return math.sqrt((p[0]-q[0])**2 + (p[1]-q[1])**2 + (p[2]-q[2])**2)

# Fuerza bruta 3D

def closest_brute_3d(pts):
    best = float('inf')
    pair = None
    for i in range(len(pts)):
        for j in range(i+1, len(pts)):
            d = distancia_3d(pts[i], pts[j])
            if d < best:
                best, pair = d, (pts[i], pts[j])
    return pair, best

# Divide y Vencerás 3D

```

```

def closest_dc_3d(pts):
    pts_x = sorted(pts, key=lambda p: p[0]) # ordenar por x una sola vez
    return _dc3d(pts_x)

def _dc3d(pts_x):
    n = len(pts_x)

    # Caso base
    if n <= 3:
        return closest_brute_3d(pts_x)

    # Dividir por la mediana en x
    mid = n // 2
    mid_x = pts_x[mid][0]

    left_pair, left_dist = _dc3d(pts_x[:mid])
    right_pair, right_dist = _dc3d(pts_x[mid:])

    # Mejor de las dos mitades
    if left_dist <= right_dist:
        best_pair, delta = left_pair, left_dist
    else:
        best_pair, delta = right_pair, right_dist

    # Franja de cruce
    # Puntos dentro de la losa de grosor 2δ alrededor de mid_x
    franja = [p for p in pts_x if abs(p[0] - mid_x) < delta]

    # Ordenar la franja por y
    franja.sort(key=lambda p: p[1])

    # Comparar cada punto con vecinos dentro del cubo δ×δ×δ
    for i in range(len(franja)):
        j = i + 1
        while j < len(franja) and (franja[j][1] - franja[i][1]) < delta:
            d = distancia_3d(franja[i], franja[j])
            if d < delta:
                delta = d
                best_pair = (franja[i], franja[j])
            j += 1

    return best_pair, delta

# Ejemplo

pts = [(1122, 6175, 300),
        (135, 4076, 890),
        (7296, 2741, 500),
        (1100, 6200, 310),
        (4000, 5000, 750)]

pair, dist = closest_dc_3d(pts)
print(f"Par más cercano: {pair}")
print(f"Distancia: {dist:.2f}")

```

Par más cercano: ((1122, 6175, 300), (1100, 6200, 310))
 Distancia: 34.77